

T.C. MARMARA ÜNİVERSİTESİ TEKNOLOJİ FAKÜLTESİ

2025-2026 EĞİTİM ÖĞRETİM YILI

BAHAR DÖNEMİ BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

MİKROİŞLEMCİLER DERSİ PROJE RAPOR DÖKÜMANI

PROJE ADI: 8086 Assembly ile Gerçek Zamanlı Hafıza Oyunu

HAZIRLAYAN : 170425821- NİSANUR ŞAHİN

PROJE DEPOSU (GitHub): Proje GitHub Dosyası

1. **Proje Özeti**

Bu çalışma, Intel 8086 mikroişlemci mimarisi üzerinde düşük seviyeli programlama teknikleri kullanılarak tasarlanmış, gerçek zamanlı bir Bellek (Memory) Oyunu projesidir. Projenin temel amacı, kısıtlı sistem kaynakları ve doğrudan donanım etkileşimi ile kullanıcı odaklı, hatasız ve performanslı bir yazılım geliştirme sürecini sergilemektir.

1.1 Projenin Kapsamı ve Amacı

Oyun, 16 birimden oluşan (4x4) bir oyun tahtası üzerinde, rastgele dizilmiş 8 farklı karakter çiftinin bulunması esasına dayanır. Proje süresince aşağıdaki temel bilgisayar mühendisliği kavramları üzerinde durulmuştur:

- Doğrudan Bellek Erişimi: Veri segmentinde tanımlanan dizilerin (array) indeksleme yöntemleri ile manipüle edilmesi.
- Gerçek Zamanlı Süreç Yönetimi: Kullanıcı klavyeden giriş yaparken, eş zamanlı olarak arka planda sistem saatinin izlenmesi ve görsel geri bildirim sağlanması.
- BIOS ve DOS Kesme (Interrupt) Programlama: Standart giriş/çıkış fonksiyonlarının ötesine geçilerek, ekran rengi, imleç konumu ve donanım saati gibi bileşenlerin doğrudan kontrolü.

1.2 Oyunun Akışı

Yazılım başlatıldığında kullanıcıyı özel tasarlanmış bir logo ve geliştirici bilgileri içeren bir giriş ekranı karşılar. Oyun başladığında, kullanıcı 0-F (Hexadecimal) arası girişlerle iki farklı kart seçer. Yazılımın karar mekanizması şu adımları izler:

1. Doğrulama: Kullanıcının seçtiği indeksin sınırlar dahilinde olup olmadığı ve kartın daha önce açılıp açılmadığı kontrol edilir.
2. Karşılaştırma: İki kartın bellekteki ASCII değerleri karşılaştırılır.
3. Görsel Güncelleme: Eşleşme başarılı ise kartlar kalıcı olarak açık kalır (Yeşil renk); başarısız ise kullanıcıya ceza puanı (süre eksilmesi) verilerek kartlar tekrar kapatılır (Açık Mavi renk).
4. Bitiş Koşulları: Tüm çiftler bulunduğunda "Kazanma", hamle sayısı tükendiğinde veya süre sıfırlandığında "Kaybetme" senaryoları tetiklenir.

---

2. Teknik Özellikler

- **Mimari:** x86 (16-bit Real Mode)
- **Dil:** Assembly (MASM/TASM uyumlu)

- **Bellek Modeli:** .MODEL SMALL (Kod ve veri segmentlerinin ayrımı)
- **Grafik Arayüzü:** Metin tabanlı (DOS Mode 03h, 80x25 renkli metin)

---

3. Sistem Tasarımı ve Yazılım Mimarisi

3.1 Veri Segmenti Yapılandırması (.DATA)

Oyunun mantıksal çerçevesi, bellekteki verilerin dinamik olarak manipüle edilmesi üzerine kurulmuştur. Veri segmenti, oyunun "statik hafızasını" temsil eden şu temel bileşenlerden oluşur:

- **gizli_kartlar:** Oyunun çözüm kümesini içeren 16 byte'lık bir veri dizisidir. Arka planda harf çiftlerinin ('A'-'H') konumlarını saklar. Kullanıcı bir seçim yaptığında, bu dizideki ilgili indeks okunarak eşleşme kontrolü yapılır.
- **acik_kartlar:** Kullanıcının görsel arayüzünü temsil eden maskeleme dizisidir. Başlangıçta tüm elemanlar '*' karakteriyle yüklüdür ve başarılı eşleşmeler gerçekleştikçe bu karakterler ilgili harflerle güncellenir.
- **Durum Kontrol Değişkenleri (İstatistikler):** Oyunun akışını ve bitiş senaryolarını yöneten dinamik sayaçlardır:
    - hamle_sayisi: Kullanıcının stratejik sınırını belirleyen ve her tahminde azaltılan sayaçtır (Başlangıç: 10).
    - kalan_sure: Oyunun zamana karşı oynanmasını sağlayan, gerçek zamanlı olarak azalan değişkendir (Başlangıç: 60 saniye).
    - eslesen_cift: Kazanma durumunu (CMP eslesen_cift, 8) kontrol eden sayaçtır.

3.2 Temel Prosedürlerin Analizi

A) ANA_DONGU

Oyunun ana motorudur. Her döngü başında kazanma ve kaybetme koşullarını (süre, hamle, eşleşme sayısı) kontrol ederek programın akışını yönlendirir. Kullanıcıdan girdi alma ve tahtayı yeniden render etme süreçlerini koordine eder.

```
ANA_DONGU:
;Her dongu basinda oyunun bitip bitmedigini kontrol ediyoruz
    CMP eslesen_cift, 8
    JE KAZANDIN
    CMP hamle_sayisi, 0
    JLE KAYBETTIN
    CMP kalan_sure, 0
    JLE KAYBETTIN

    CALL EKRANI_TEMIZLE

    ; Baslik (Camgobegi)
    LEA DX, msg_baslik
    MOV BL, 0BH
    CALL PRINT_STR_RENKLI

    ; Istatistikler
    LEA DX, msg_stats
    MOV BL, 07H
    CALL PRINT_STR_RENKLI
    MOV AL, hamle_sayisi
    MOV BL, 0AH
    CALL SAYI_YAZDIR

    ; Zamanlayici
    ; Sure 10 saniyenin altina dustugunde oyuncuyu uyarmak icin rengi saridan kirmiziya (0CH) ceviriyoruz.
    LEA DX, msg_time
    MOV BL, 07H
    CALL PRINT_STR_RENKLI
    MOV AL, kalan_sure
    MOV BL, 0EH      ; Sari
    CMP AL, 10
    JAE SURE_RENGI_TAMAM
    MOV BL, 0CH      ; 10 Saniyenin altinda Kirmizi
SURE_RENGI_TAMAM:
```

B) KARAKTER_OKU

1. Klavye Kontrolü: INT 16H / AH=01H ile klavye sürekli kontrol edilir.
2. Zaman Senkronizasyonu: Klavye girişi beklenirken INT 21H / AH=2CH kesmesiyle sistem saati okunur.
3. Güncelleme: Saniye değişimi tespit edildiğinde kalan_sure azaltılır ve imleç anlık olarak üst bilgi çubuğuna taşınarak süre güncellenir.

```
KARAKTER_OKU PROC
BEKLE_TUS:
    MOV AH, 01H
    INT 16H
    JNZ TUS_AL

    MOV AH, 2CH
    INT 21H
    CMP DH, son_saniye
    JE BEKLE_TUS

    MOV son_saniye, DH
    CMP kalan_sure, 0
    JLE ZAMAN_DOLDU
    DEC kalan_sure

    MOV AH, 03H
    MOV BH, 0
    INT 10H
    MOV imlec_satir, DH
    MOV imlec_sutun, DL

    MOV AH, 02H
    MOV BH, 0
    MOV DH, 2
    MOV DL, 32
    INT 10H

    MOV AL, kalan_sure
    MOV BL, 0EH
    CMP AL, 10
    JAE RENK_AYARLANDI
    MOV BL, 0CH
RENK_AYARLANDI:
    CALL SAYI_YAZDIR

    MOV AH, 02H
    MOV BH, 0
    MOV DH, imlec_satir
    MOV DL, imlec_sutun
    INT 10H

    JMP BEKLE_TUS

ZAMAN_DOLDU:
    POP AX
    JMP KAYBETTIN
```

C) PRINT_STR_RENKLI

BIOS kesmelerini doğrudan kullanarak yüksek performanslı bir görsel çıktı sağlar. INT 10H / AH=09H kesmesini kullanarak metinleri sadece ASCII olarak değil, aynı zamanda renk öznitelikleriyle birlikte ekrana basar. Bu sayede kartların durumuna göre (kapalı, açık, hatalı) farklı renk tonları atanabilir.

```
PRINT_STR_RENKLI PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX
    PUSH SI

    MOV SI, DX
    ; Baslangic kursorunu 1 kez al
    MOV AH, 03H
    MOV BH, 0
    INT 10H

STR_DONGU:
    MOV AL, [SI]
    CMP AL, '$'
    JE STR_BITIR

    CMP AL, 10
    JE STR_YENI_SATIR
    CMP AL, 13
    JE STR_SATIR_BASI

    ; Karakteri Rengiyle Birlikte Bas
    MOV AH, 09H
    MOV BH, 0
    MOV CX, 1
    INT 10H

    ; Kursoru manuel hesaplayarak hiz kazan
    INC DL
    CMP DL, 80
    JB KURSOR_SET
    MOV DL, 0
    INC DH
    JMP KURSOR_SET

STR_YENI_SATIR:
    INC DH
    JMP KURSOR_SET

STR_SATIR_BASI:
    MOV DL, 0

KURSOR_SET:
    MOV AH, 02H
    MOV BH, 0
    INT 10H
```

D) INDEKSE_CEVIR

Kullanıcıdan alınan Hexadecimal ASCII karakterini (0-9, A-F) matematiksel bir modelle 0-15 aralığındaki dizi indeksine dönüştürür. Bu prosedür, karakterin tipine göre farklı çıkartma işlemleri (0-9 için 48, A-F için 55 çıkartılması gibi) yaparak ham girdiyi bellek adresleme modlarına uygun hale getirir.

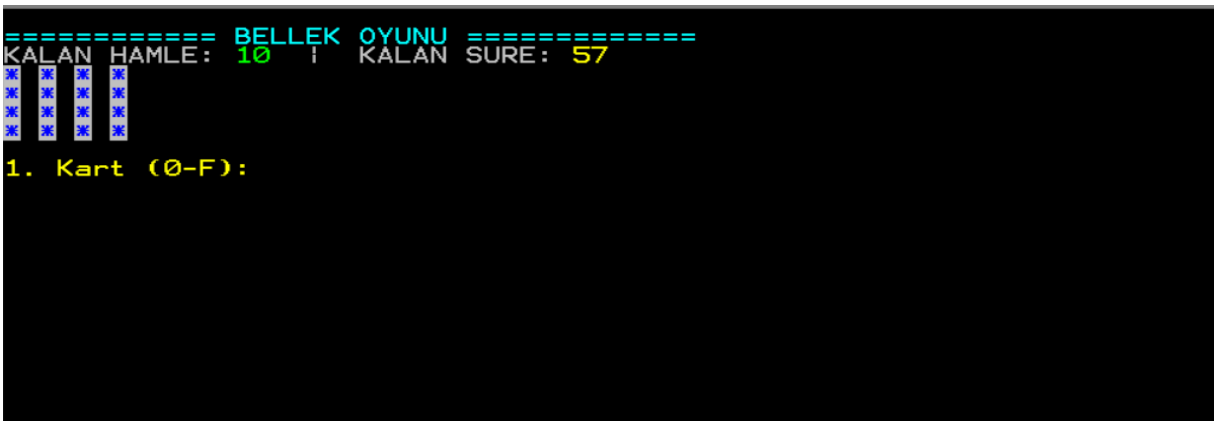
```
INDEKSE_CEVIR PROC
    MOV AH, 0
    CMP AL, '0'
    JB HATA_IND
    CMP AL, '9'
    JBE RAKAM
    CMP AL, 'A'
    JB HATA_IND
    CMP AL, 'F'
    JBE B_HARF
    CMP AL, 'a'
    JB HATA_IND
    CMP AL, 'f'
    JBE K_HARF
HATA_IND:
    MOV AH, 1
    RET
RAKAM: SUB AL, 48
    RET
B_HARF: SUB AL, 55
    RET
K_HARF: SUB AL, 87
    RET
INDEKSE_CEVIR ENDP
```

3.3 Ekran Çıktıları

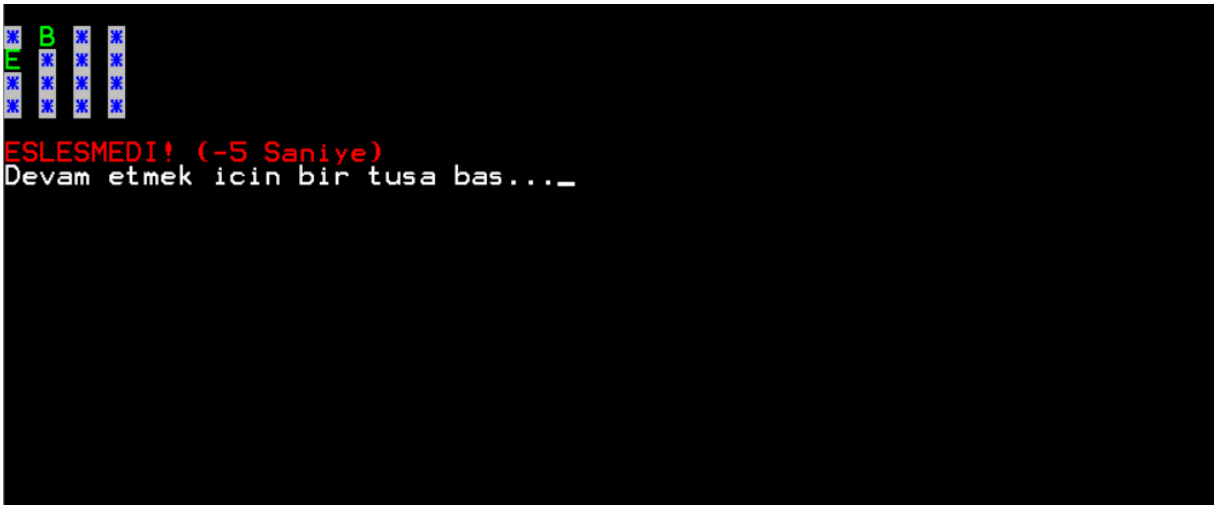
a)Giriş Ekranı



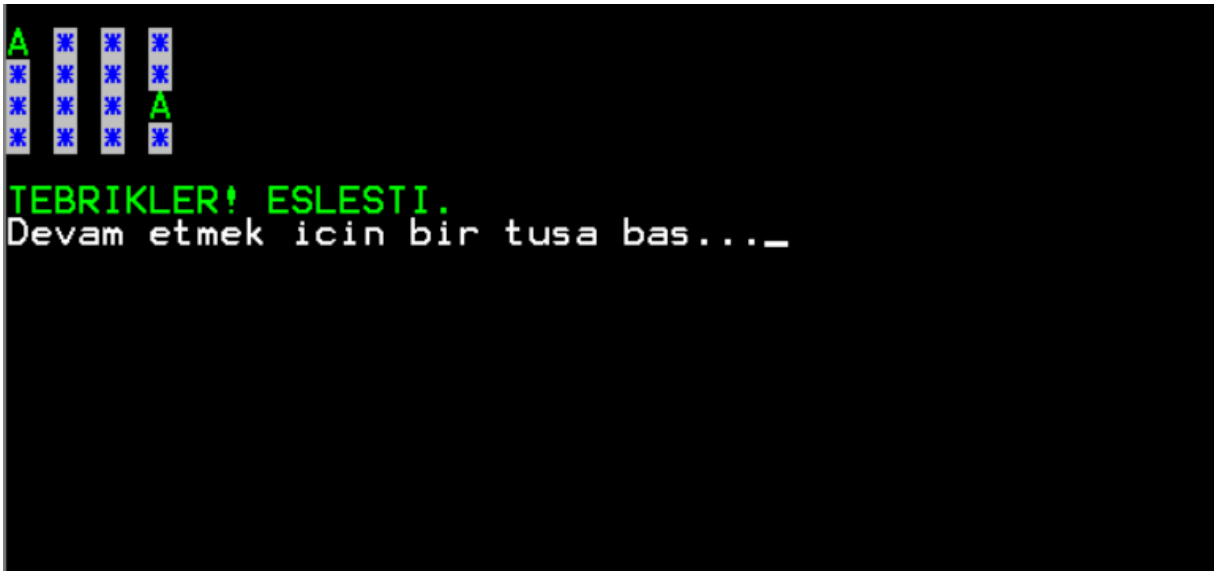
b)Oyunun Genel Tasarımı



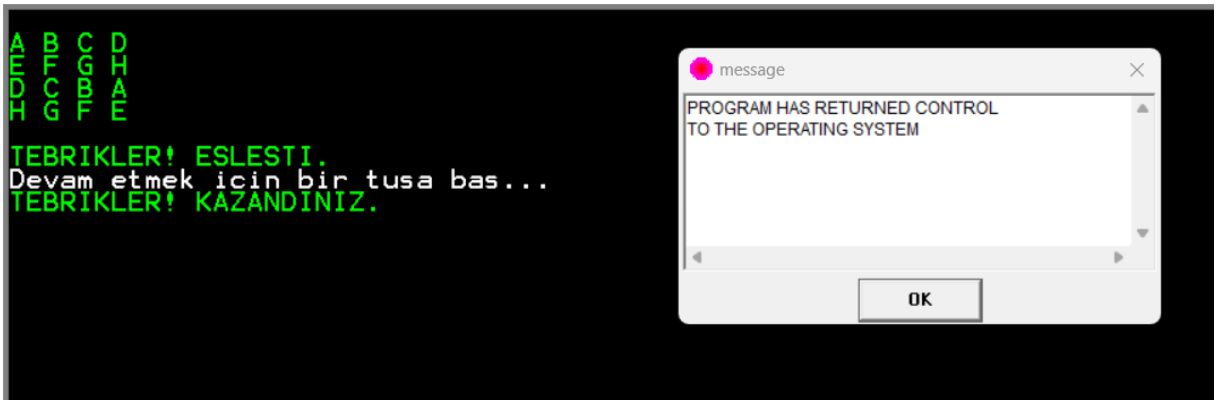
c) Seçilen Kartların Eşleşmemesi Durumu



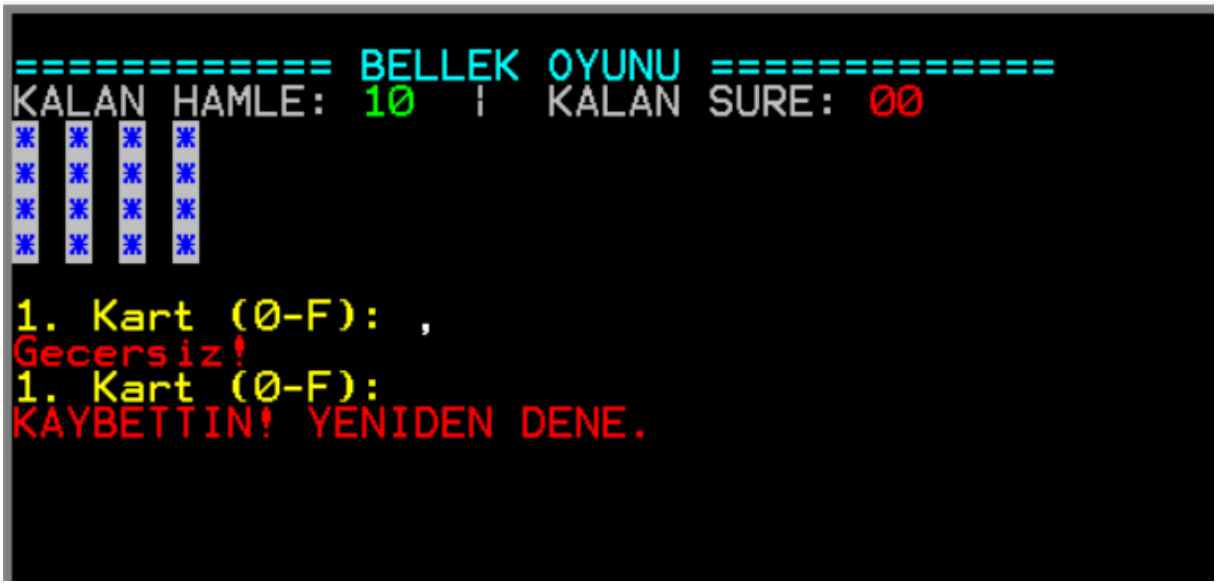
d)Seçilen Kartların Eşleşmesi Durumu



e)Oyunu Kazanma Durumu



f)Oyunu Kaybetme Durumu



---

4. Uygulama Detayları ve Algoritma Analizi

4.1 Renk Yönetimi ve Görsel Geri Bildirim Mekanizması

Projenin görsel arayüzü, standart metin çıktılarının ötesinde, kullanıcı deneyimini (UX) optimize etmek amacıyla **BIOS INT 10H / AH=09H** kesmesiyle zenginleştirilmiştir. Bu kesme, ekrana sadece karakter basmakla kalmaz, aynı zamanda her karakter için bir "Attribute Byte" atamamıza olanak tanır.

- **Dinamik Renk Atamaları:** * **Kapalı Kartlar (79H):** Arka planı açık mavi, metni beyaz olarak ayarlanmış öznitelik baytı ile kullanıcıya seçilebilir alanlar gösterilir.
    - **Doğru Eşleşmeler (0AH):** Başarıyla bulunan harf çiftleri canlı yeşil renge dönüştürülerek görsel onay sağlanır.
    - **Kritik Durum Uyarıları (0CH):** kalan_sure değişkeni 10 saniyenin altına düştüğünde, karşılaştırma komutları (CMP) ve koşullu dallanmalar (JAE/JB) kullanılarak sayaç rengi sarıdan kırmızıya dönüştürülür. Bu, kullanıcıda psikolojik bir aciliyet hissi yaratmak için kullanılan bir tasarım tercihidir.

4.2 Gerçek Zamanlı Zamanlayıcı Nasıl Çalışıyor?

Oyunun en kritik teknik zorluğu, kullanıcı klavyeden giriş yaparken arka planda sürenin eş zamanlı olarak akmasını sağlamaktır. Geleneksel yaklaşımda bu iki işlem birbirini bloklar; yani program ya klavyeyi dinler ya da saati kontrol eder. Bu projede ise her ikisi tek bir döngü içinde çözülmüştür.

Zamanlayıcının çalışma prensibi şu adımlardan oluşur:

1. Adım — Klavye Yoklama (Polling): INT 16H / AH=01H kesmesi, klavyede bekleyen tuş olup olmadığını programı durdurmadan kontrol eder. Eğer tuş yoksa program beklemek yerine saati kontrol etmeye devam eder. Bu sayede program hiçbir zaman "donmaz".

2. Adım — Sistem Saatinin Okunması: INT 21H / AH=2CH kesmesi ile işletim sisteminin o anki saniye değeri DH registerına okunur. Bu değer, bir önceki döngüde kaydedilen son_saniye değişkeniyle karşılaştırılır.

3. Adım — Değişim Tespiti: İki değer birbirinden farklıysa tam olarak bir saniye geçmiş demektir. Bu noktada kalan_sure bir azaltılır ve son_saniye yeni değerle güncellenir.

4. Adım — Kısmi Ekran Güncellemesi: Ekranın tamamını silip yeniden çizmek görüntüde titreme (flickering) yaratır. Bunun önüne geçmek için INT 10H / AH=02H ile imleç yalnızca süre sayısının bulunduğu koordinata taşınır, sadece o iki rakam güncellenir ve imleç hemen kullanıcının giriş yaptığı konuma geri döner. Tüm bu işlem milisaniyeler içinde gerçekleştiği için kullanıcı ekranda herhangi bir titreme görmez.

5. Adım — Kritik Durum Kontrolü: kalan_sure değeri 10'un altına düştüğünde renk sarıdan kırmızıya dönüştürülür. Süre sıfıra ulaşırsa stack temizlenerek doğrudan KAYBETTIN etiketine atlanır.

4.3 Oyun Mantığı ve Karar Mekanizmaları

Oyunun lojik akışı, veri bütünlüğünü korumak ve hata payını sıfırlamak üzere kurgulanmıştır:

- **Girdi Doğrulama:** Kullanıcının girdiği karakterin 0-F aralığında olup olmadığı ve daha önce açılmış bir kartı tekrar seçip seçmediği maske dizisi (acik_kartlar) üzerinden kontrol edilir.
- **Eşleşme Algoritması:** Seçilen iki kartın bellek adreslerindeki değerler AL ve AH registerlarına alınarak CMP komutuyla karşılaştırılır.
- **Ceza ve Ödül Sistemi:** * Eşleşme durumunda eslesen_cift sayacı artırılarak kazanma koşuluna (8) yaklaşılır.
    - Hatalı eşleşmede, kullanıcıya sadece görsel bir hata mesajı verilmez, aynı zamanda SUB kalan_sure, 5 komutuyla süre cezası uygulanarak oyunun zorluk dengesi sağlanır.

---

5. SONUÇ VE DEĞERLENDİRME

Bu proje, düşük seviyeli bir dil olan Assembly ile donanım katmanında uygulama geliştirmenin temel prensiplerini başarıyla kanıtlamıştır. Yazılımın geliştirilmesi sürecinde; koşullu dallanmalar (jump) ile program akış kontrolü, dizi manipülasyonu ile veri yönetimi ve doğrudan BIOS/DOS kesmeleri ile çevre birim kontrolü konularında derinlemesine yetkinlik kazanılmıştır.

Ayrıca, mikroişlemci mimarisindeki kayıtçı (register) yönetimi ve bellek adresleme modlarının önemi, oyunun performanslı çalışmasını sağlayan temel unsurlar olarak gözlemlenmiştir. Sonuç olarak; kısıtlı kaynaklara sahip bir sistemde hem dinamik bir zamanlayıcıyı yöneten hem de kullanıcı etkileşimini kesintisiz sağlayan stabil bir oyun motoru inşa edilmiştir. Bu çalışma, modern yüksek seviyeli dillerin temelinde yatan çalışma mantığını kavramak adına önemli bir mühendislik deneyimi sunmuştur.